Lesson 02 Demo 06

Integrating Infrastructure as Code Into the CI/CD Pipeline

Objective: To automate infrastructure deployment and enhance DevOps practices by integrating Terraform script execution into the AWS CodePipeline for efficient and consistent S3 bucket provisioning

Tools required: AWS CodePipeline, Terraform, and Git

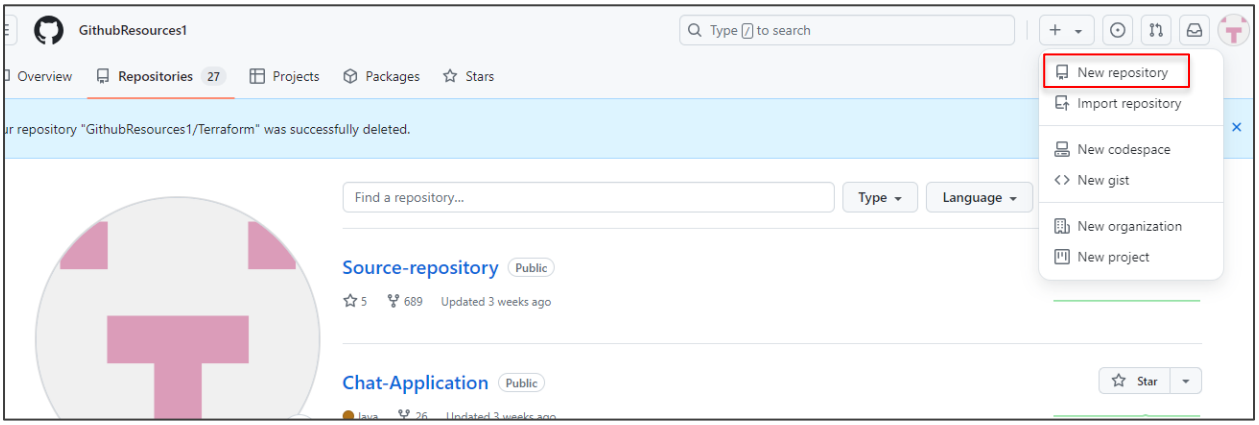
Prerequisites: None

Steps to be followed:

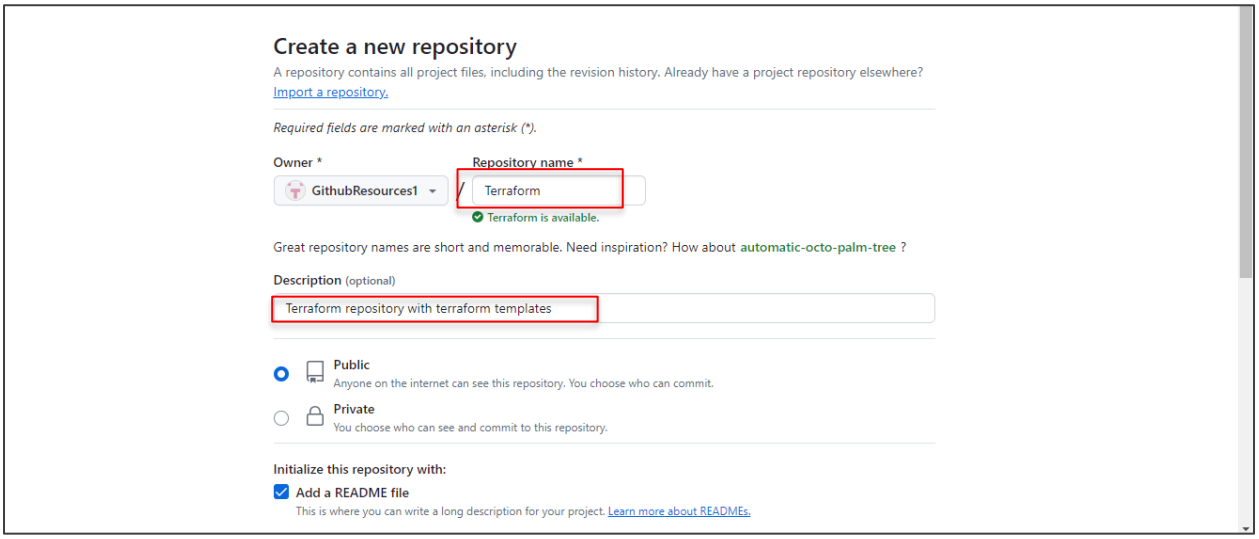
1. Configure the Git repository with the new Terraform template for deploying the S3 bucket
2. Configure CodePipeline to integrate Terraform script execution

Step 1: Configure the Git repository with the new Terraform template for deploying the S3 bucket

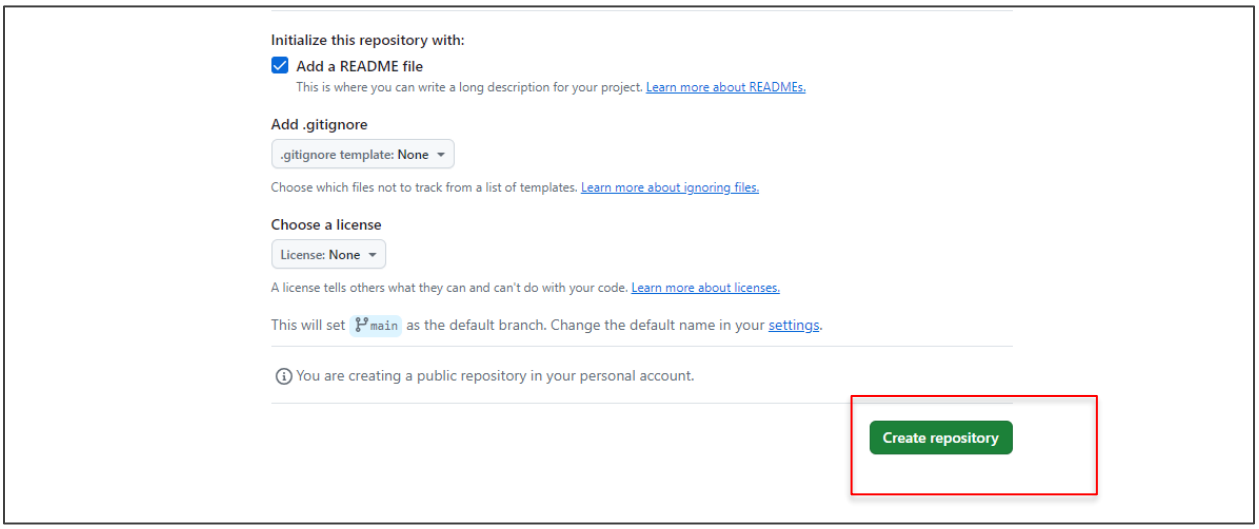
1.1 Sign in to your GitHub account and click on **New repository**



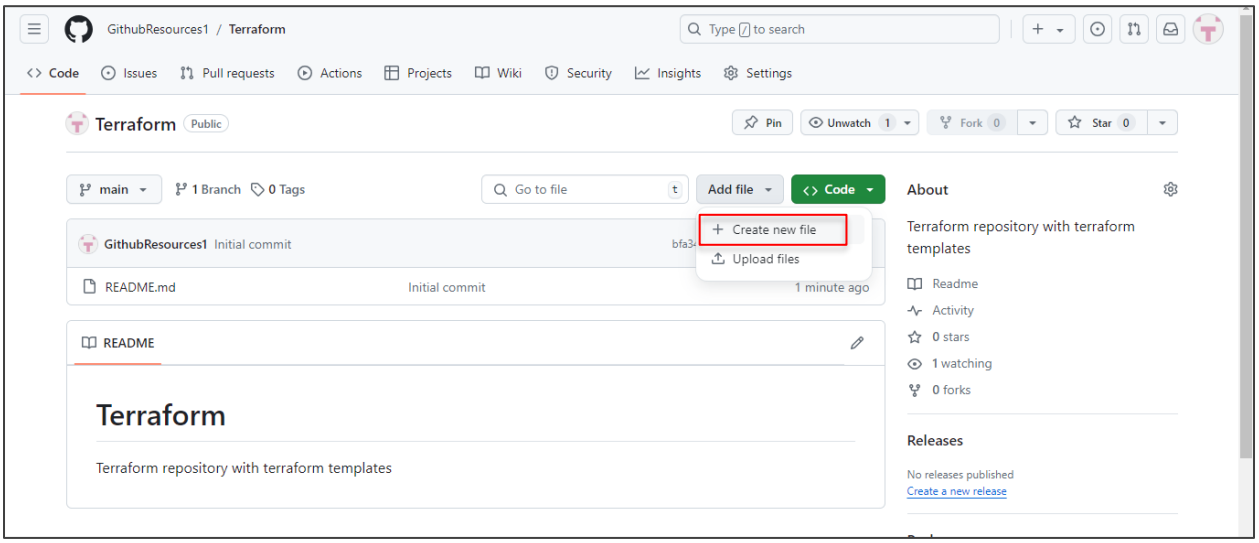
1.2 Enter the **Repository name** as **Terraform** and add the **Description** as **Terraform repository with terraform templates**



1.3 Click on the **Create repository** button



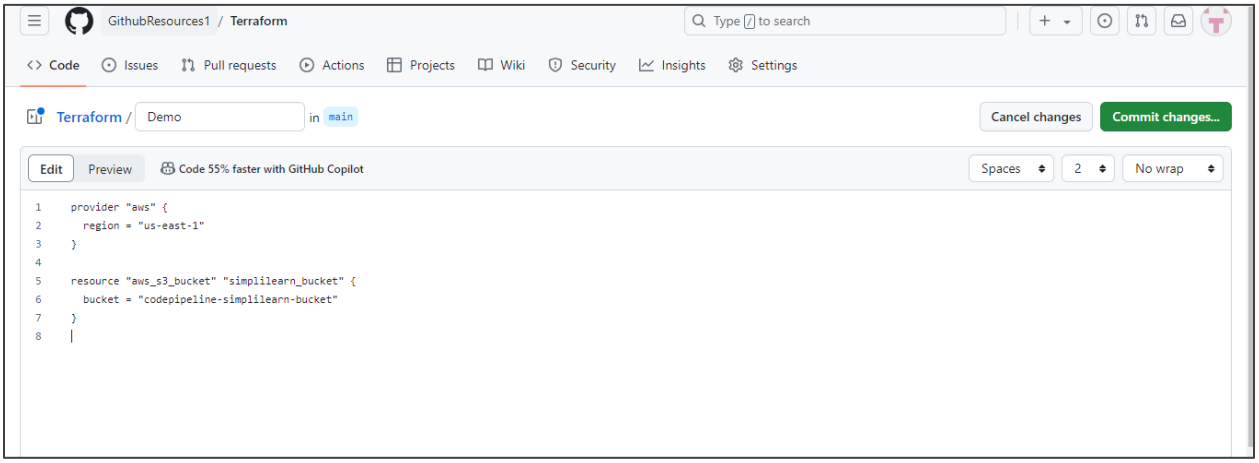
1.4 In the repository, click on **+ Create new file**



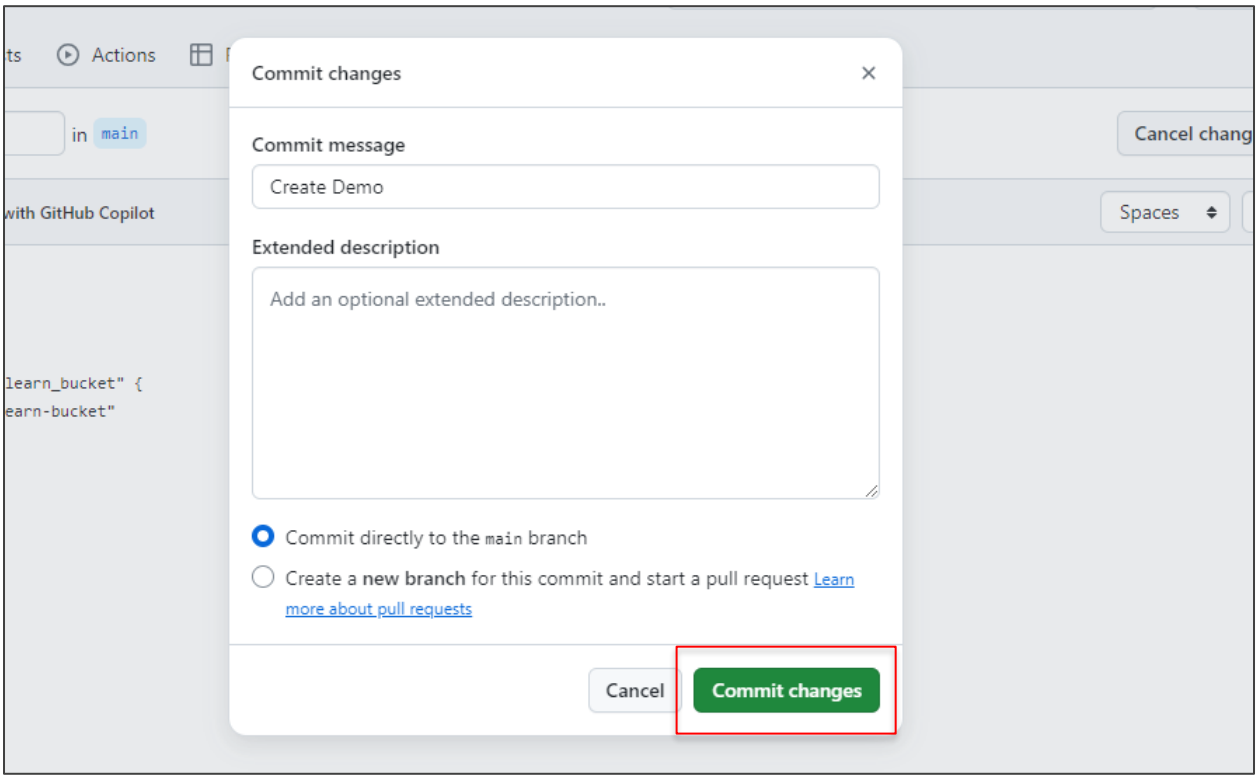
1.5 Add the following code to the file:

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "simplilearn_bucket" {
  bucket = "codepipeline-simplilearn-bucket"
}
```

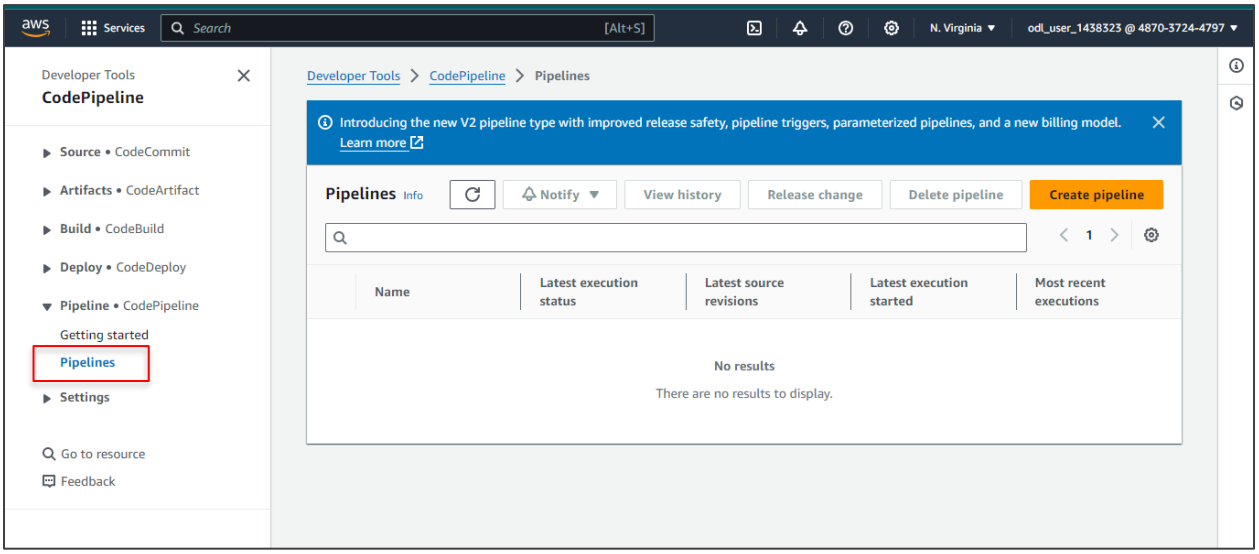


1.6 Add the **Commit message** as **Create Demo** and click on **Commit changes**

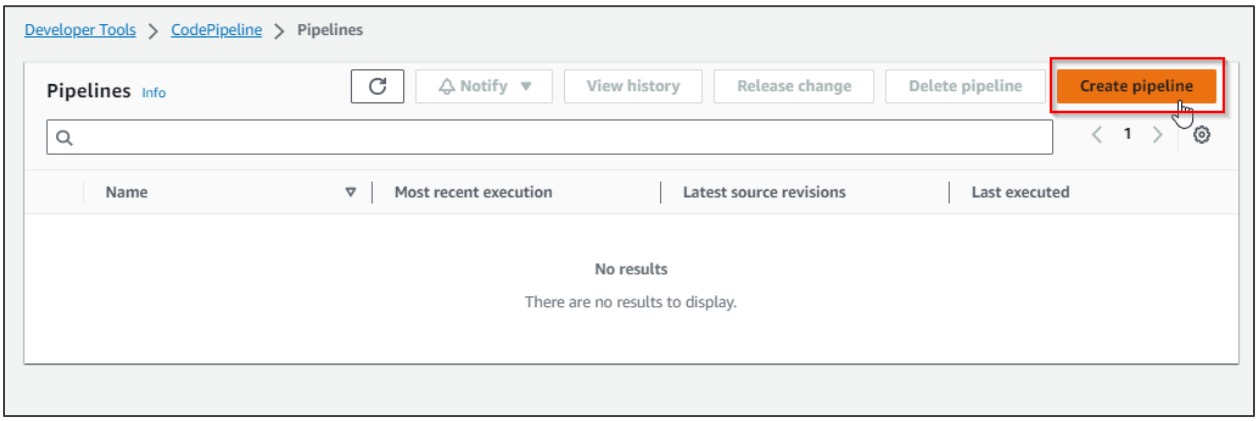


Step 2: Configure CodePipeline to integrate Terraform script execution

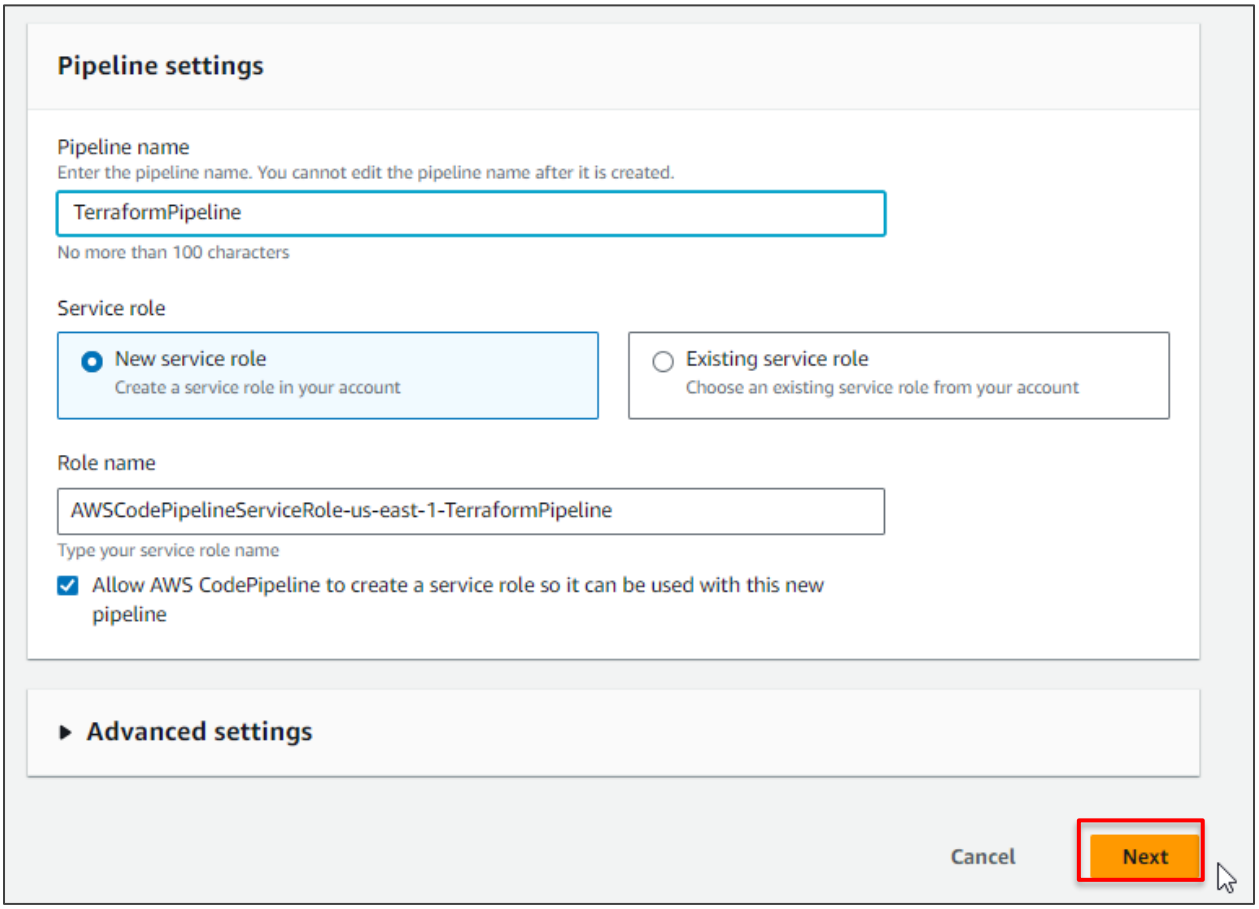
2.1 Navigate to the **CodePipeline** section and click on **Pipelines**



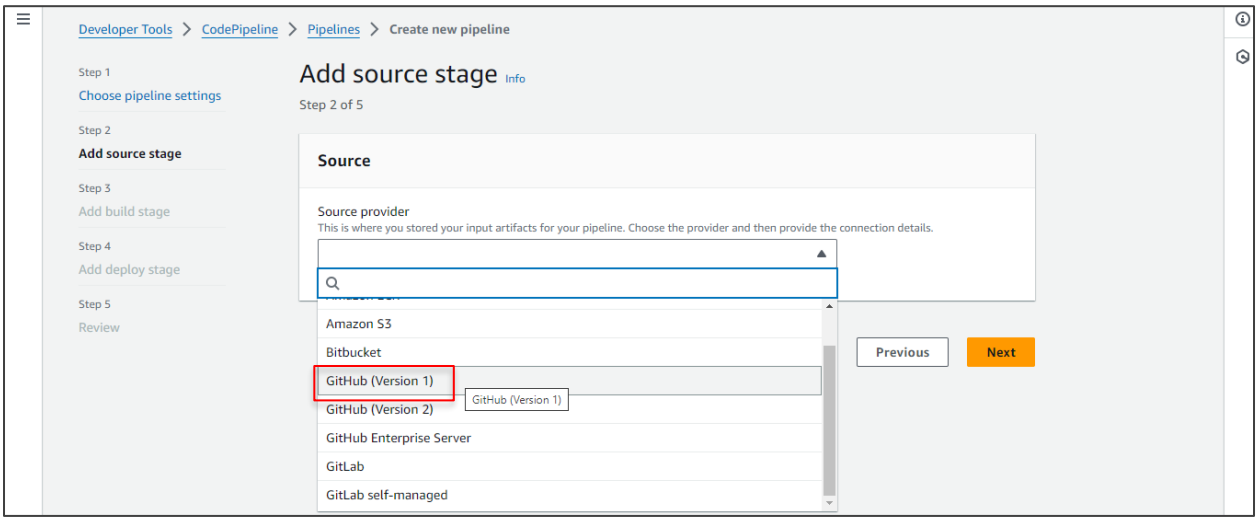
2.2 Click on the **Create pipeline** button



2.3 Enter the **Pipeline name** as **TerraformPipeline** and click on the **Next** button

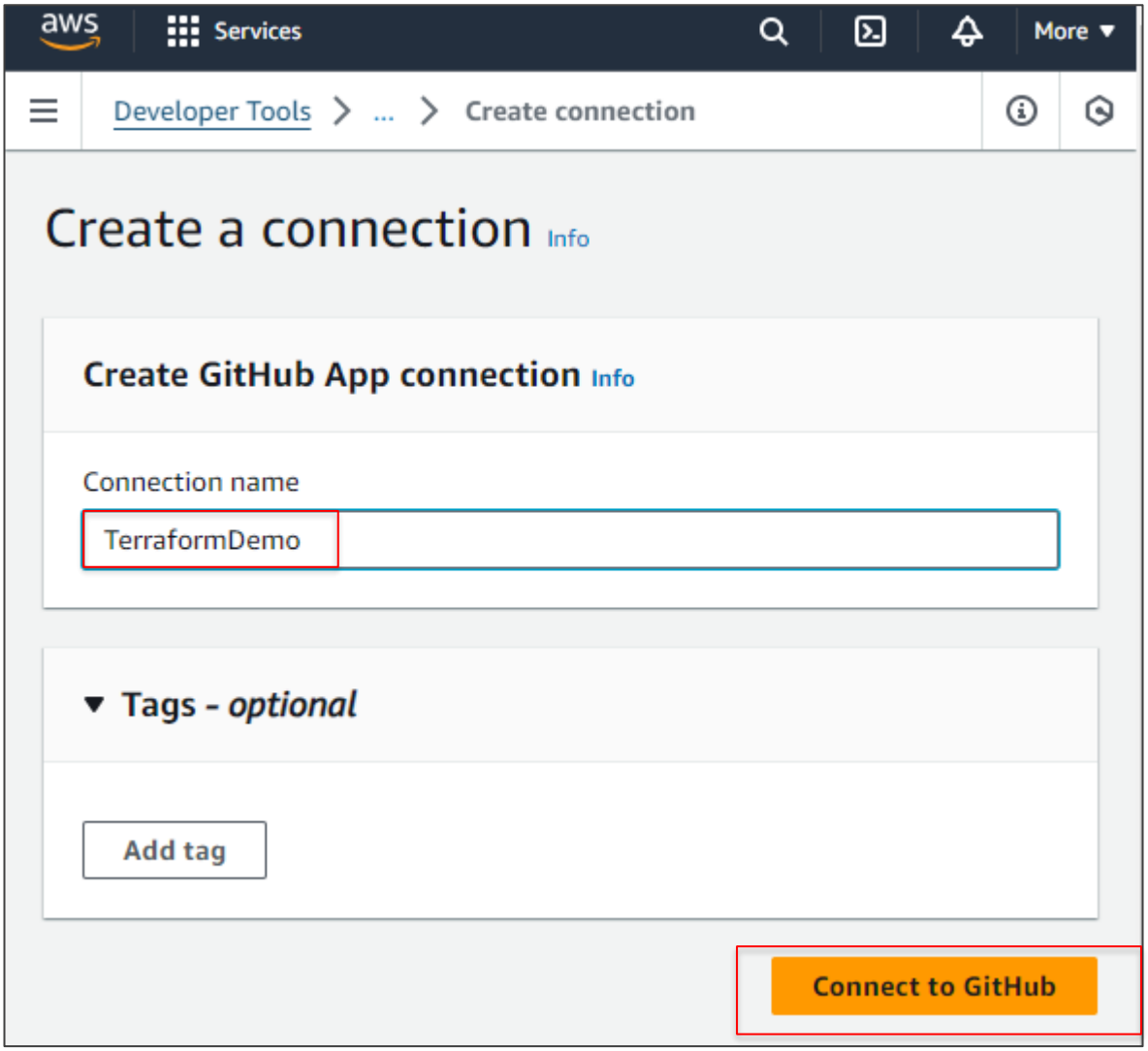


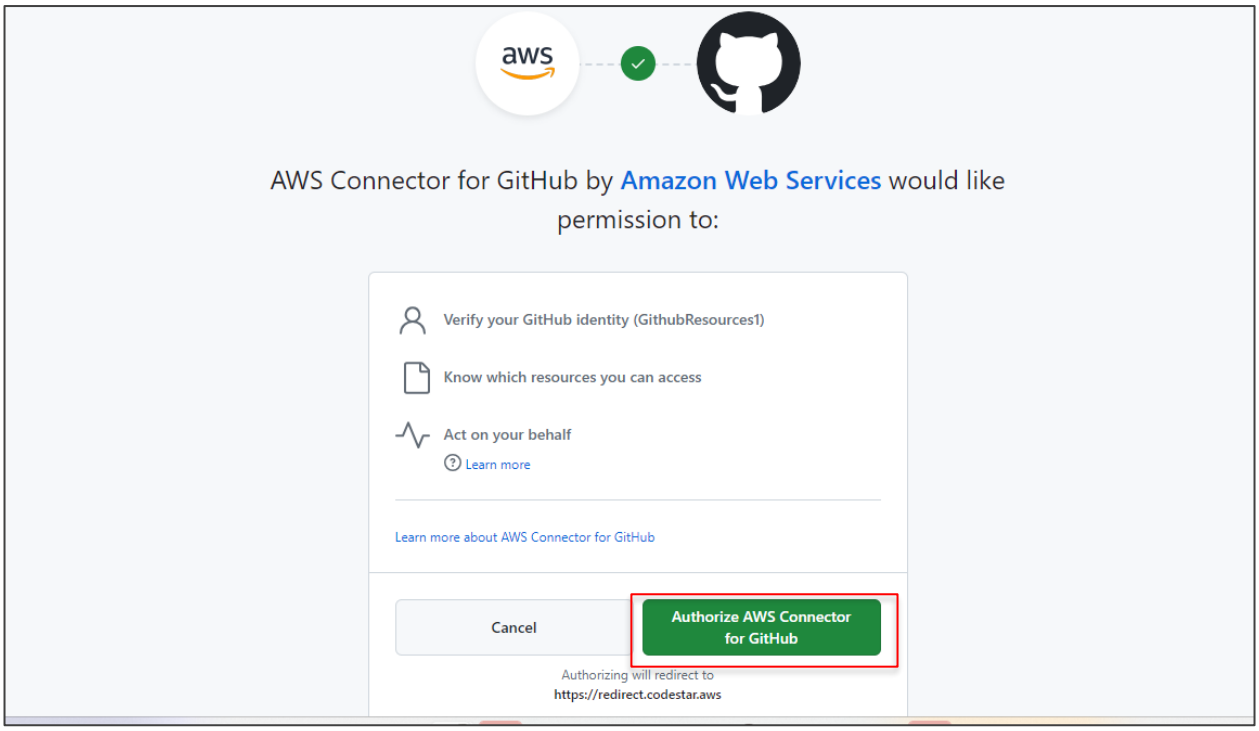
2.4 Select the **Source provider** as **GitHub (Version 1)**



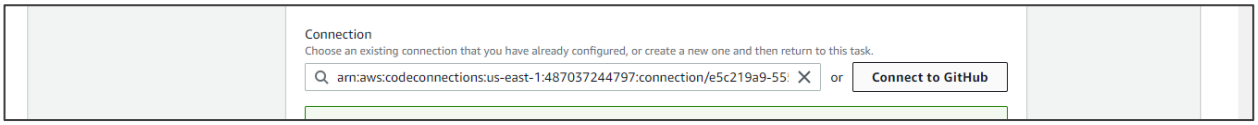
2.5 Click on **Connect to GitHub**

2.6 Enter the **Connection name** as **TerraformDemo** and click on **Connect to GitHub**. Then, click on **Authorize AWS Connector for GitHub**

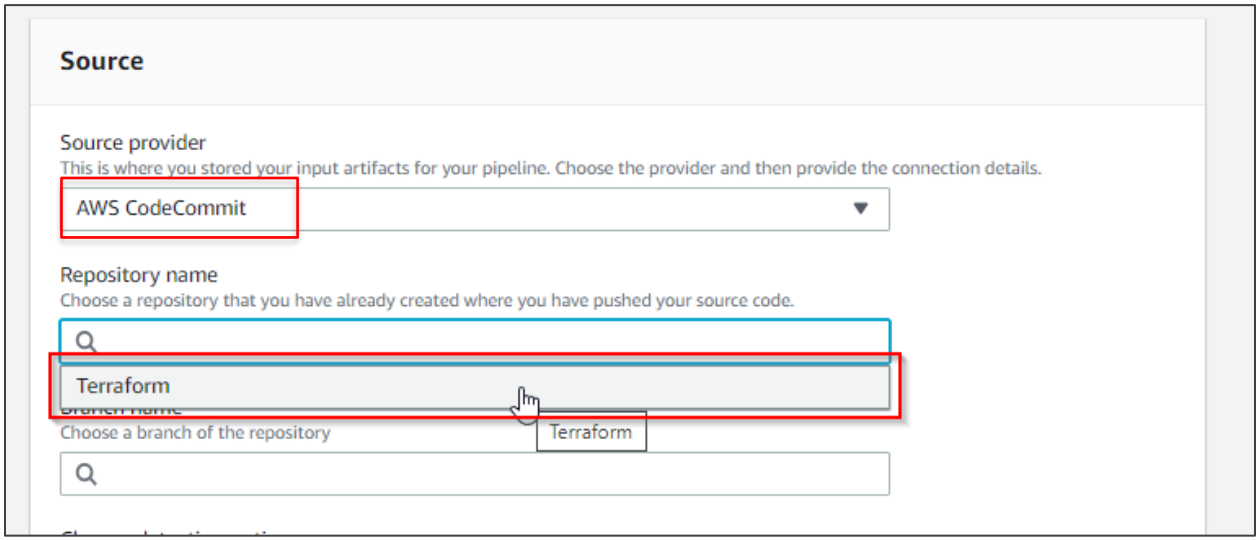


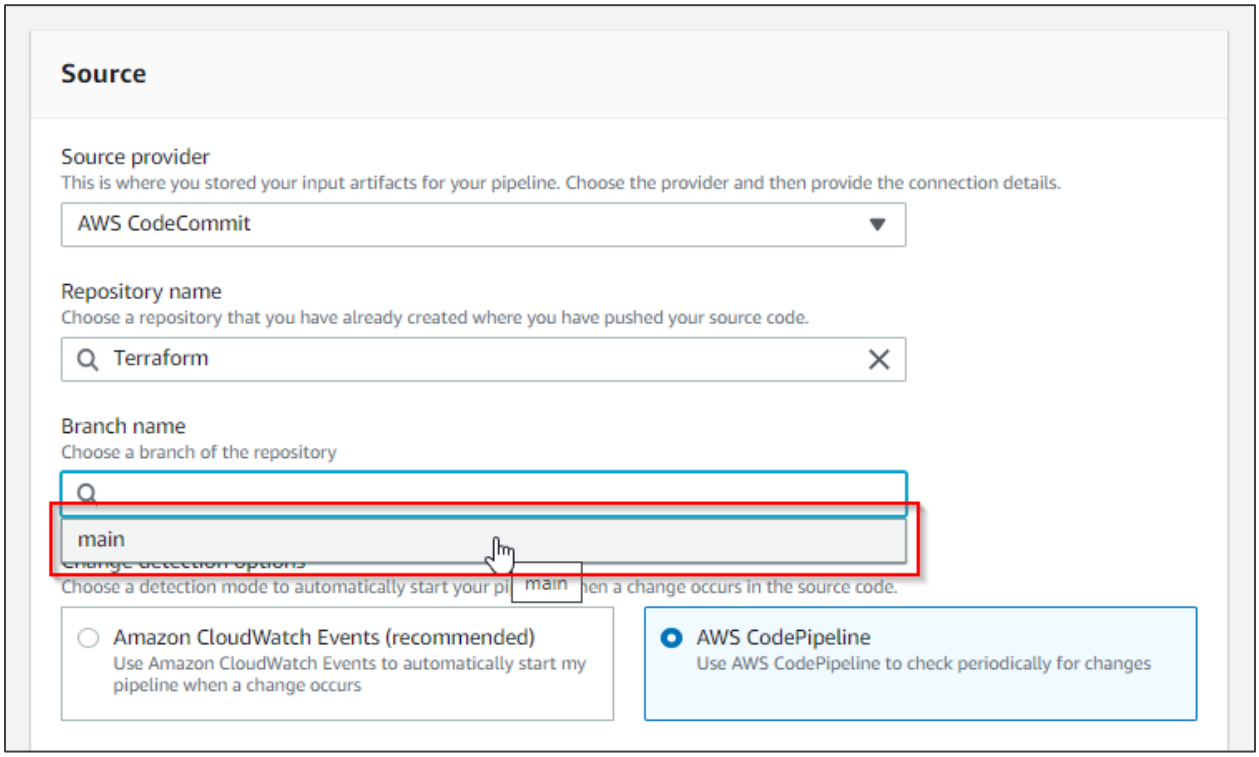


You will see that AWS is successfully connected to GitHub.

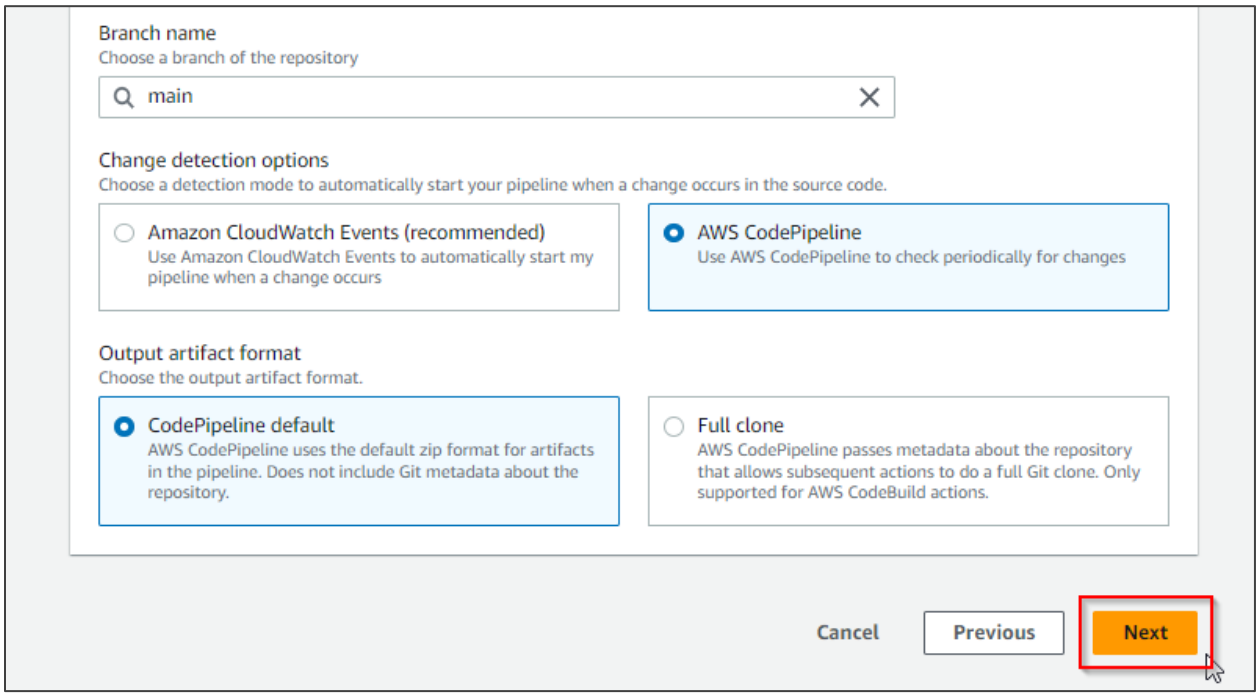


2.7 Select **AWS CodeCommit** as the **Source provider**, **Terraform** as the **Repository name**, and **main** as the **Branch name**

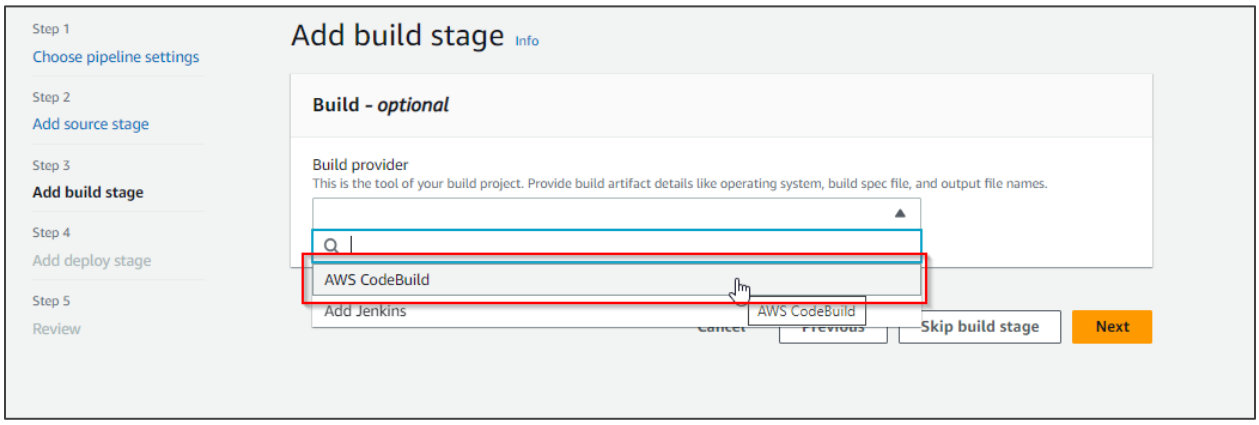




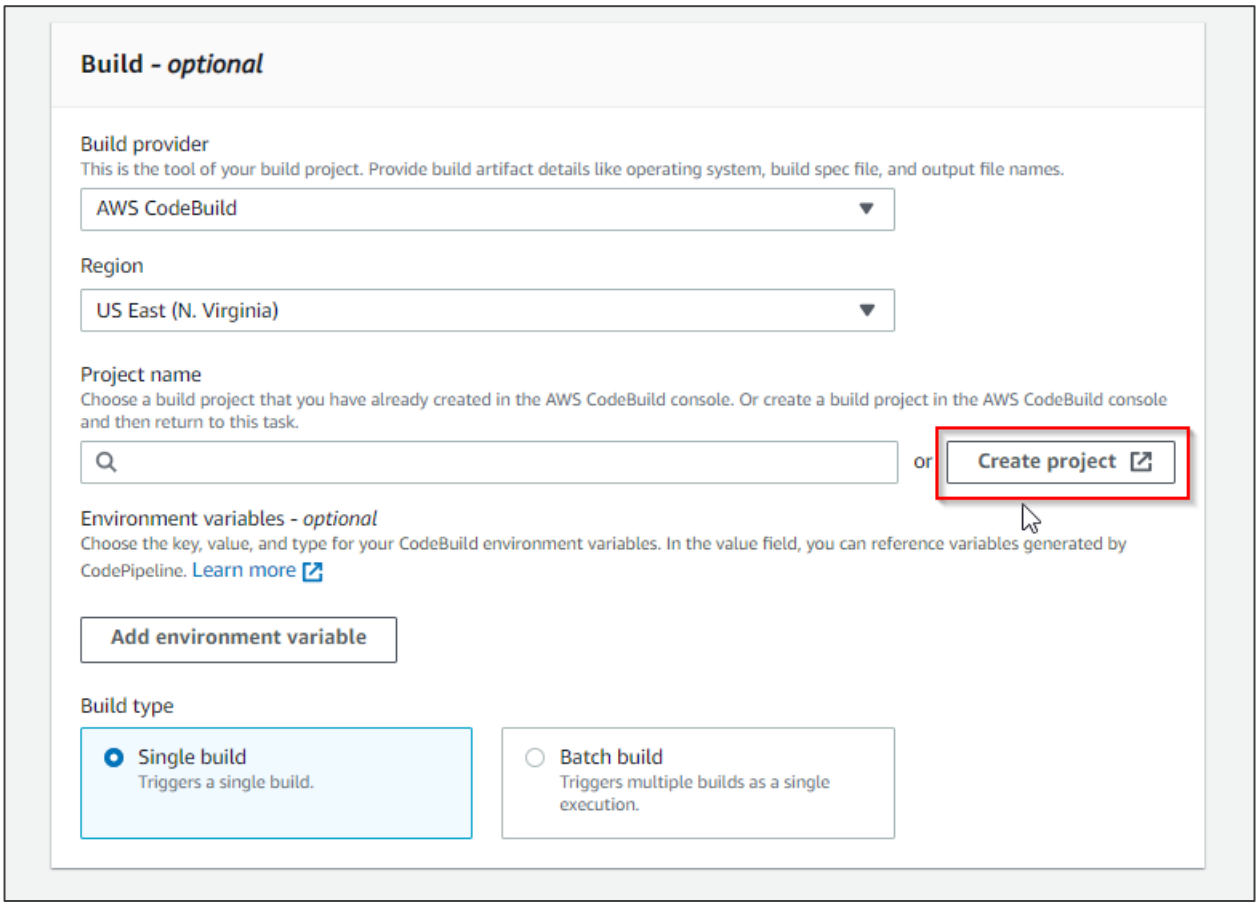
2.8 Once the details are selected, click on the **Next** button



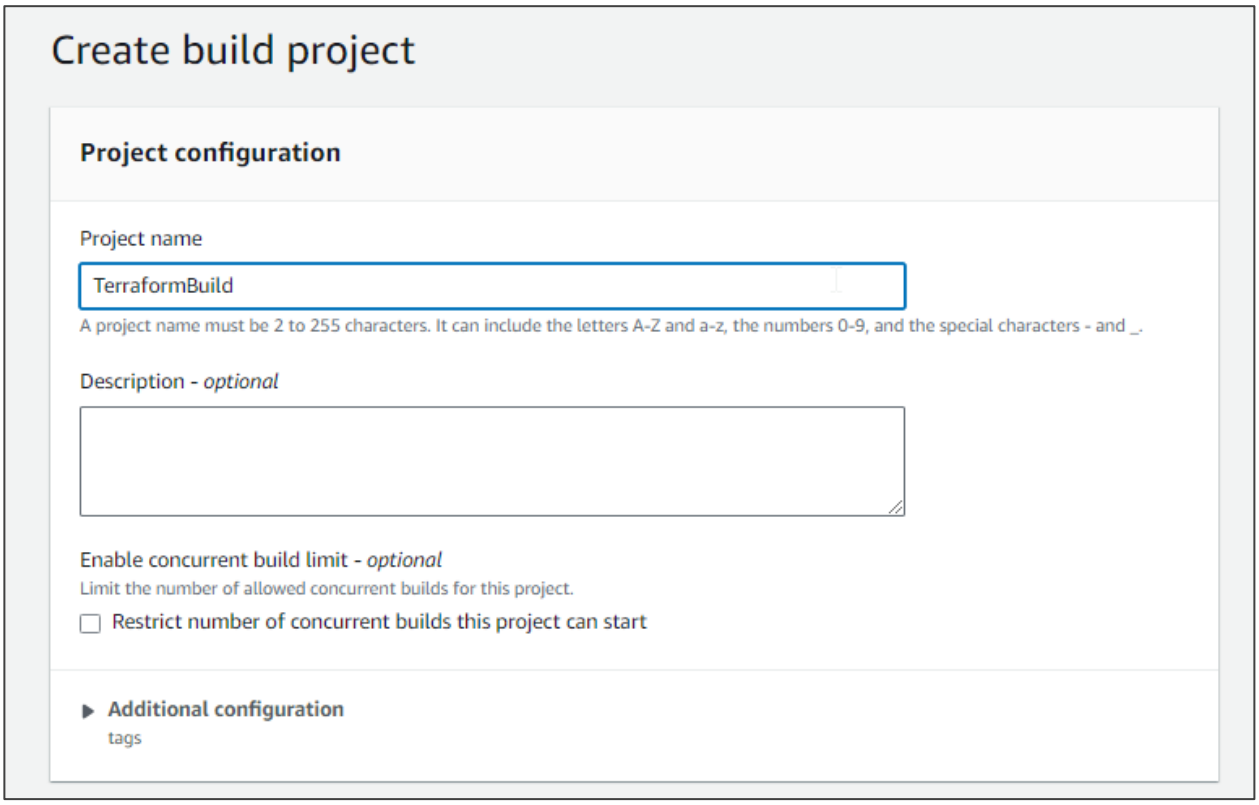
2.9 Select the **Build provider** as **AWS CodeBuild** for building and automating tests



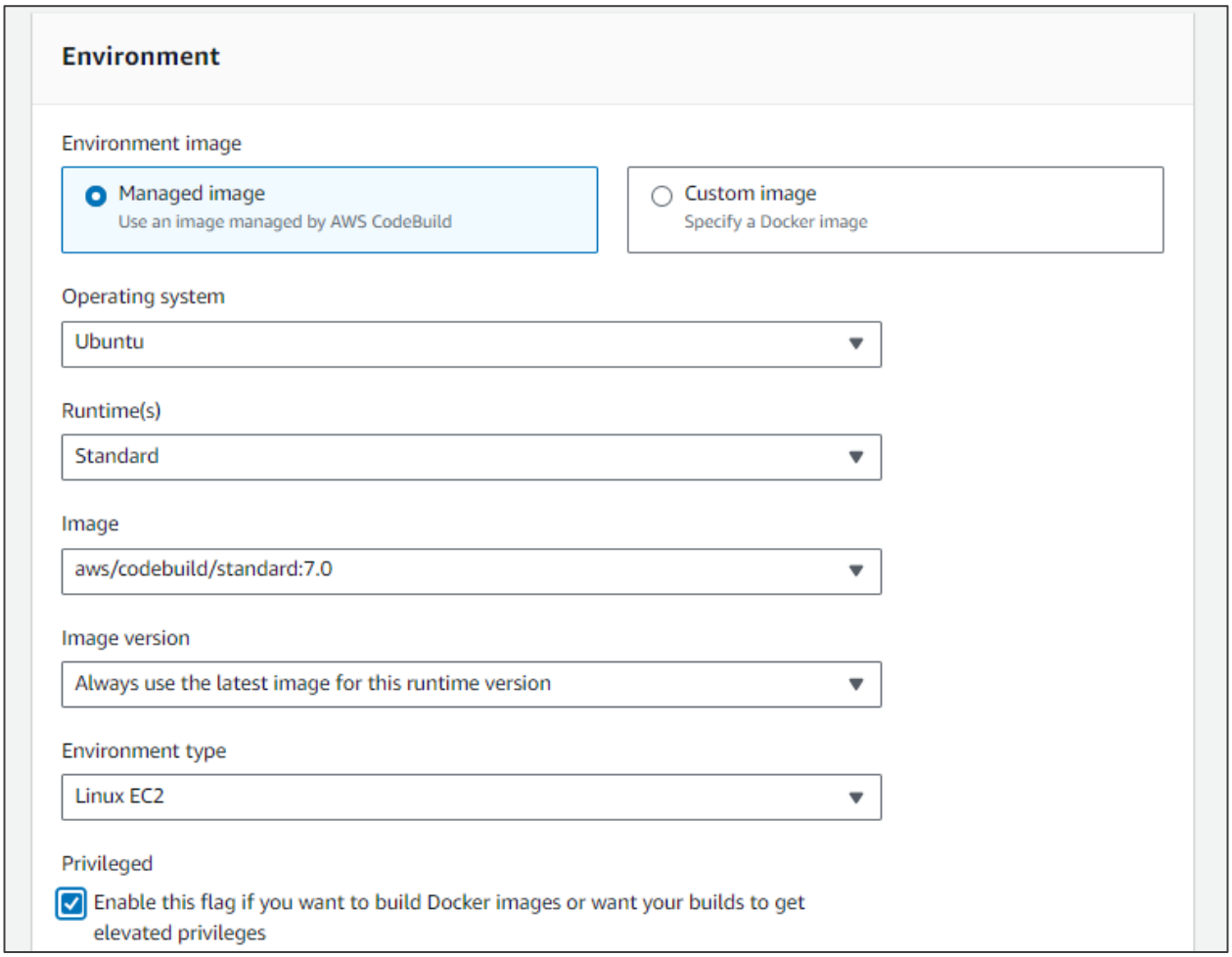
2.10 Click on the **Create project** button and a new pop-up window will appear

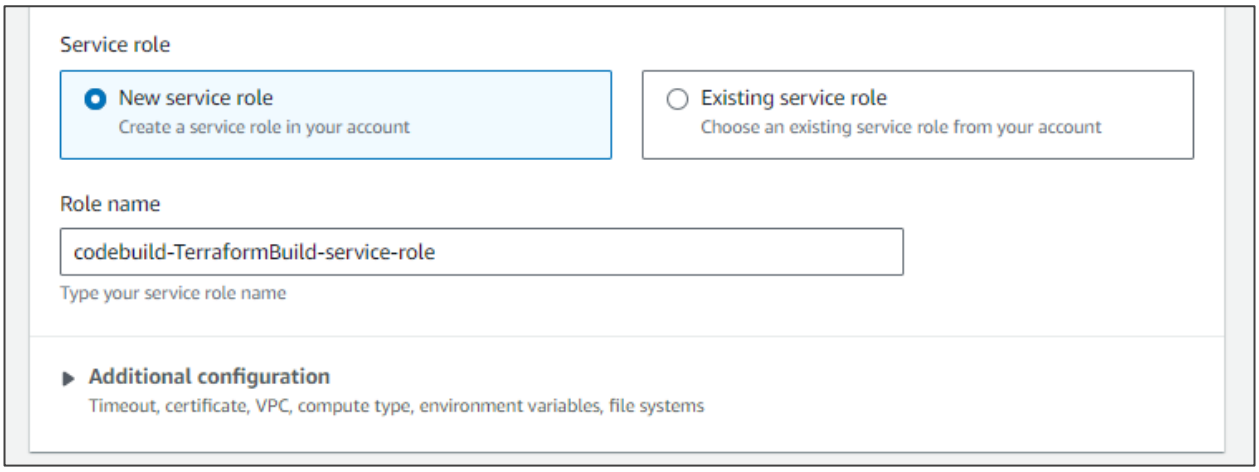


2.11 In the new window, enter the **Project name** as **TerraformBuild**

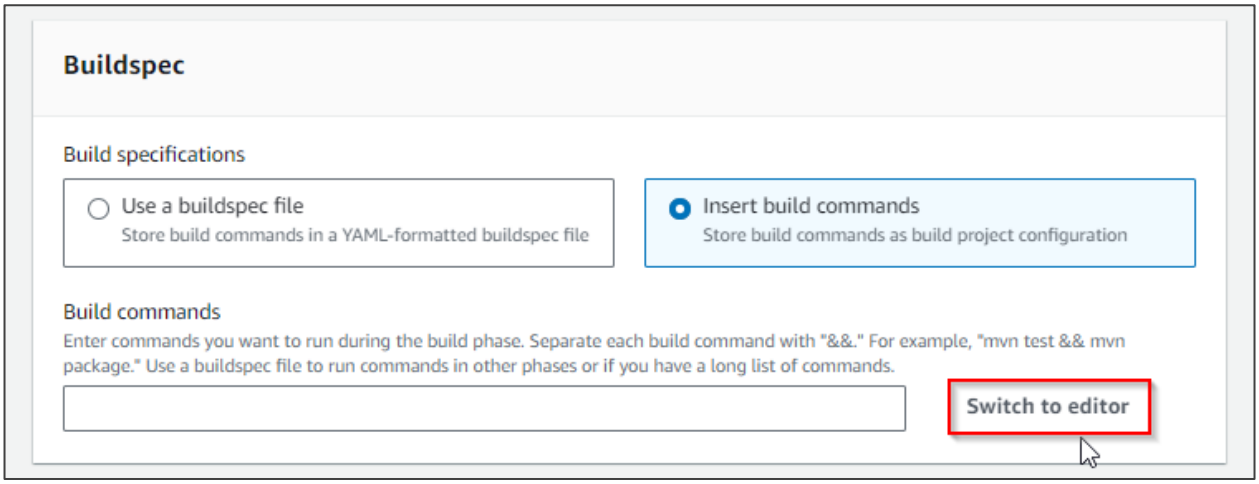


2.12 In the **Environment** section, add the required details as shown in the screenshots below:





2.13 In the **Buildspec** section, select the **Insert build commands** option and click on the **Switch to editor** button



2.14 Remove the existing build commands from the editor and enter the YAML code below:

```
version: 0.2
phases:
 install:
  commands:
   - curl -s

    https://releases.hashicorp.com/Terraform/1.3.6/Terraform_1.3.6_linux_amd64.zi
    p
     -o Terraform.zip
   - unzip Terraform.zip -d /usr/local/bin
   - chmod 755 /usr/local/bin/Terraform
 build:
  commands:
   - Terraform init
   - Terraform apply --auto-approve
artifacts:
 name: TerraformPlan
 files:
- "Terraform.tfstate*"
```

Buildspec

Build specifications

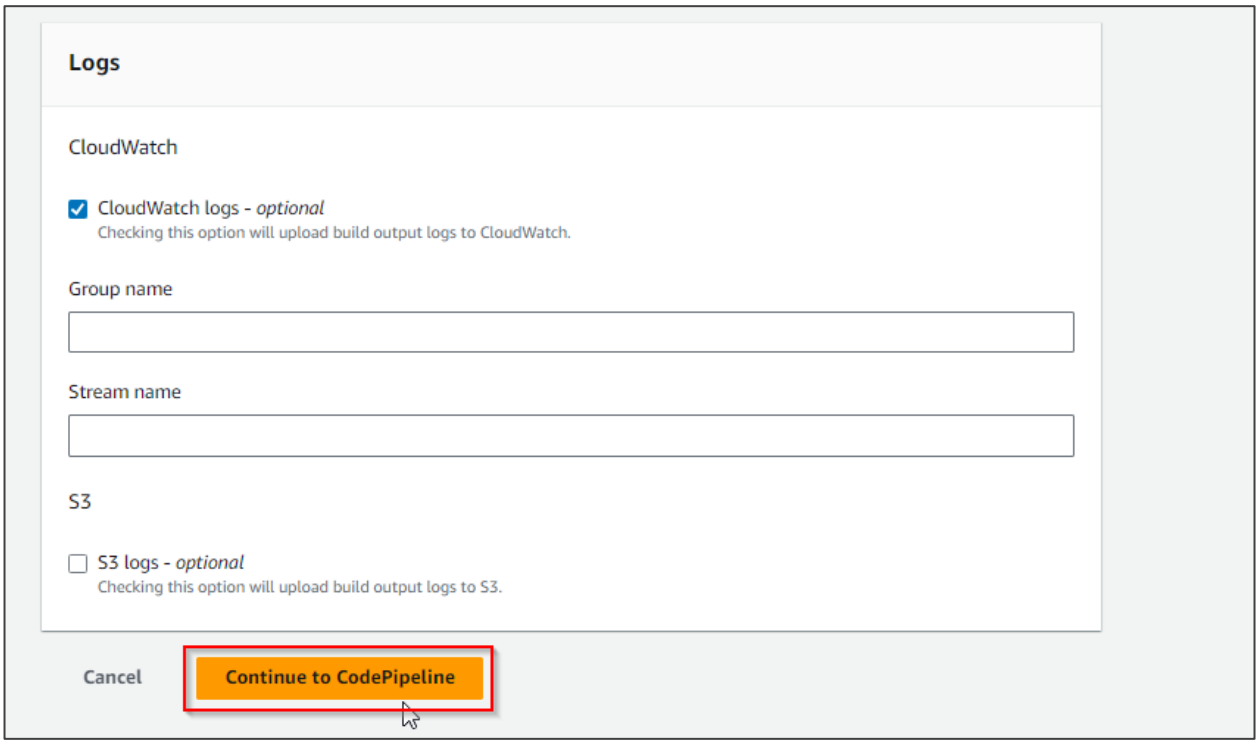
- ○ Use a buildspec file
  Store build commands in a YAML-formatted buildspec file
- ◉ Insert build commands
  Store build commands as build project configuration

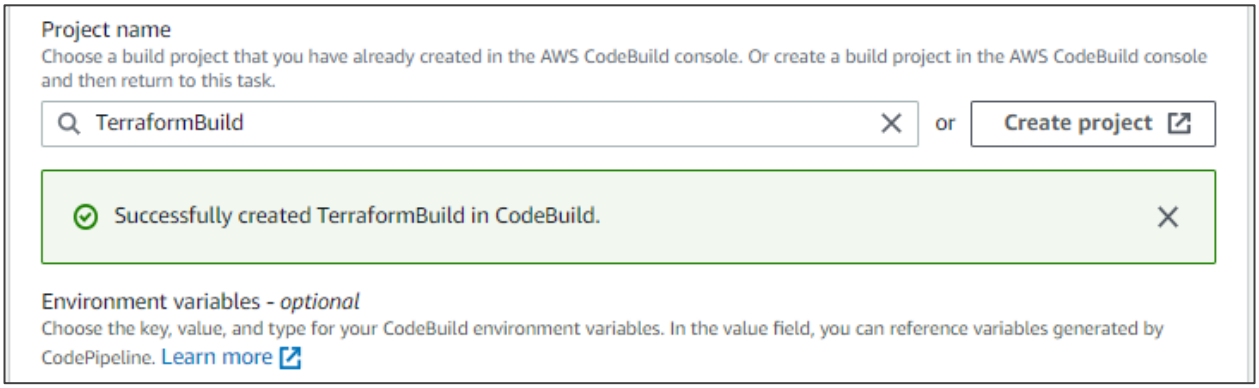
Build commands

```yaml
version: 0.2
phases:
  install:
    commands:
      - curl -s
        https://releases.hashicorp.com/terraform/1.3.6/terraform_1.3.6_linux_amd64.zip
        -o terraform.zip
      - unzip terraform.zip -d /usr/local/bin
      - chmod 755 /usr/local/bin/terraform
  build:
    commands:
      - terraform init
      - terraform apply --auto-approve
artifacts:
  name: TerraformPlan
  files:
    - "terraform.tfstate*"
```

Switch to single line

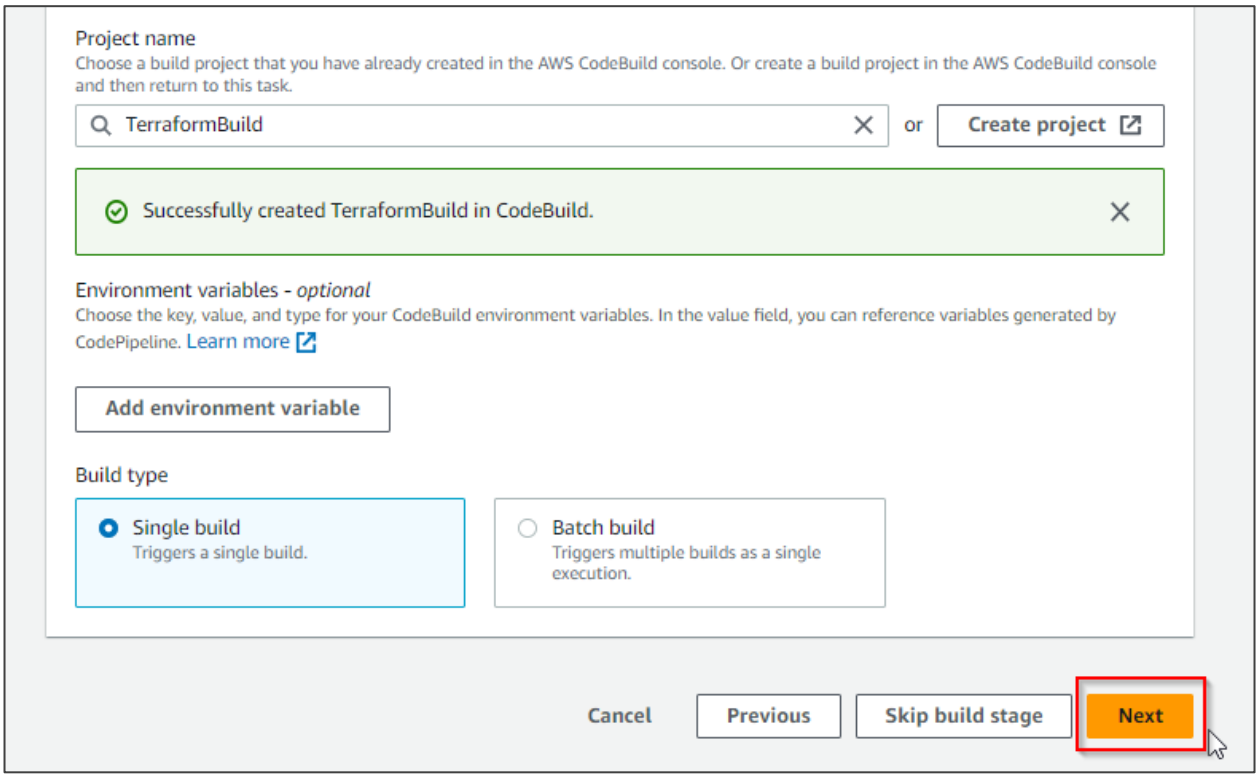
2.15 Scroll to the bottom of the page and click on the **Continue to CodePipeline** button



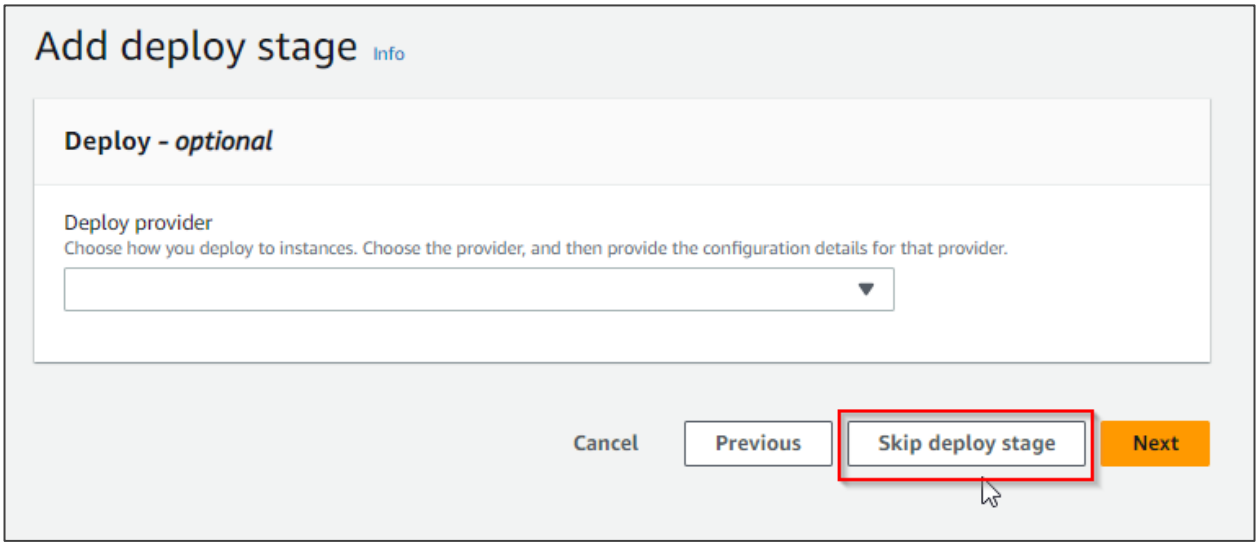


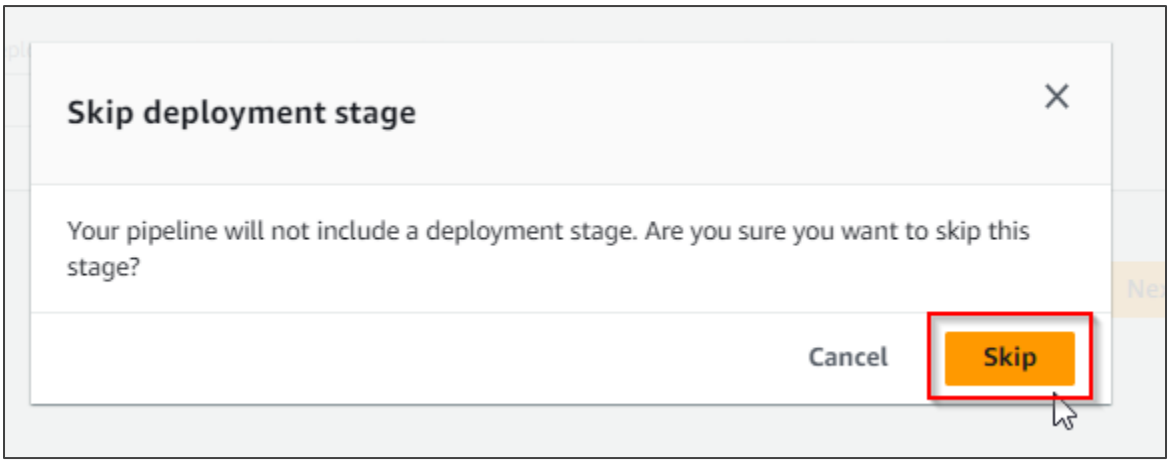
The project is successfully created.

2.16 Click on the **Next** button

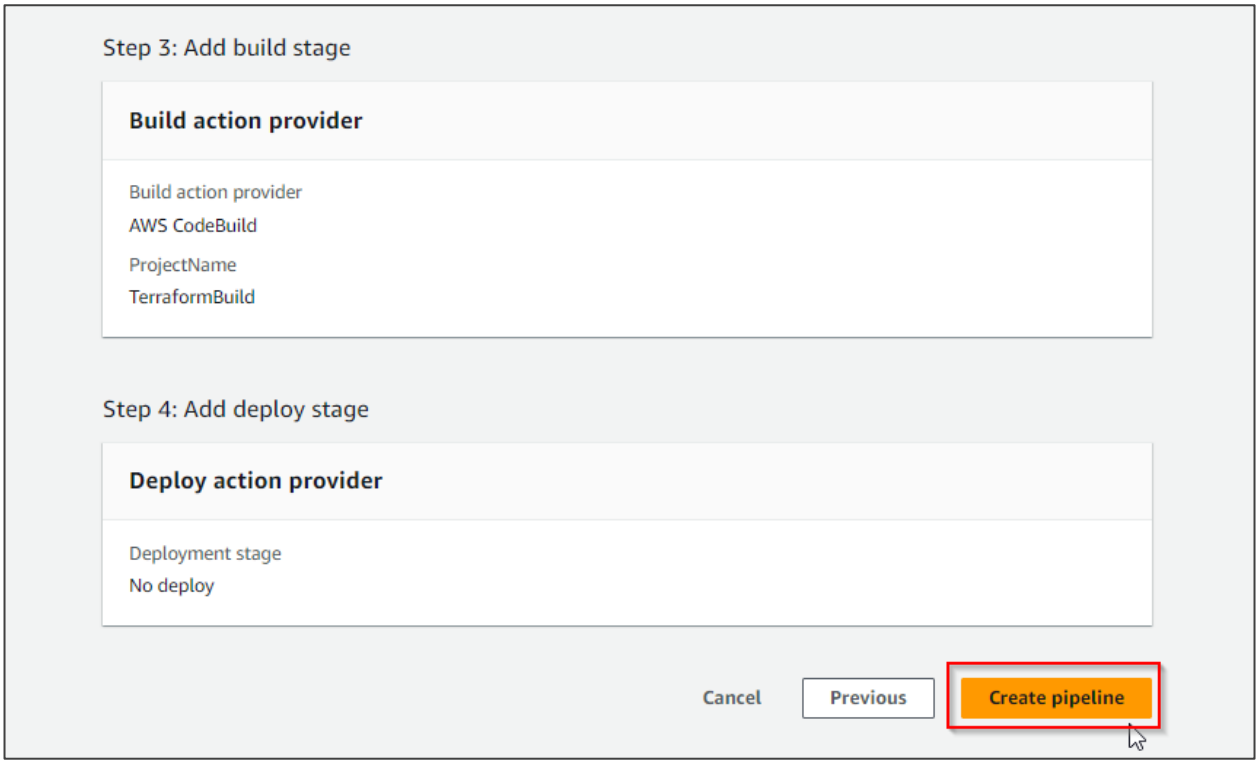


2.17 Click on the **Skip deploy stage** button and then on the **Skip** button to finalize the pipeline

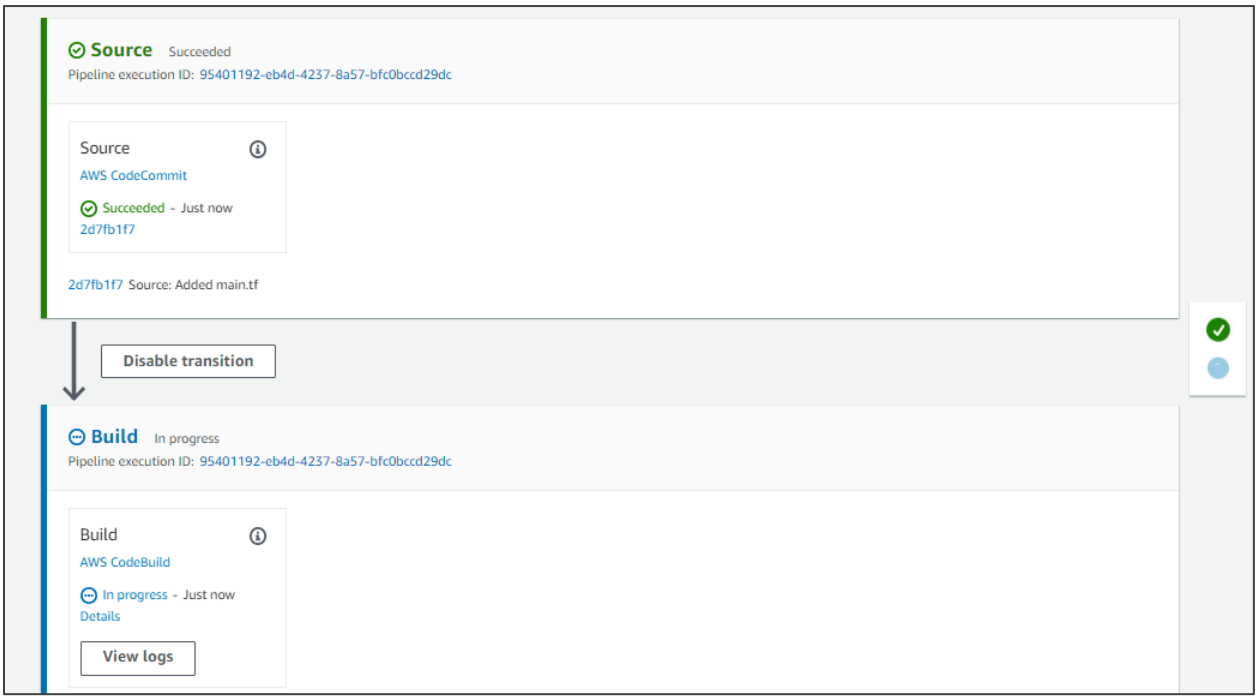




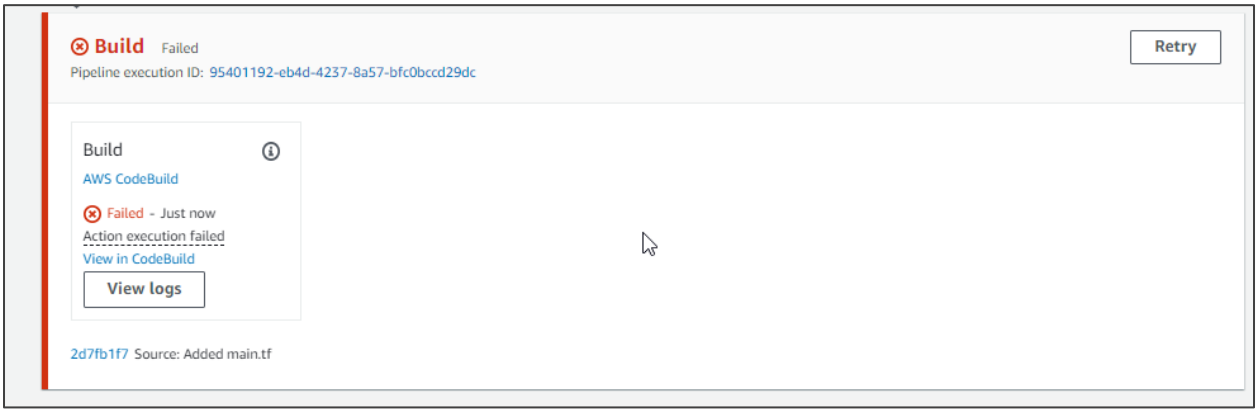
2.18 Scroll to the bottom of the page and click on the **Create pipeline** button



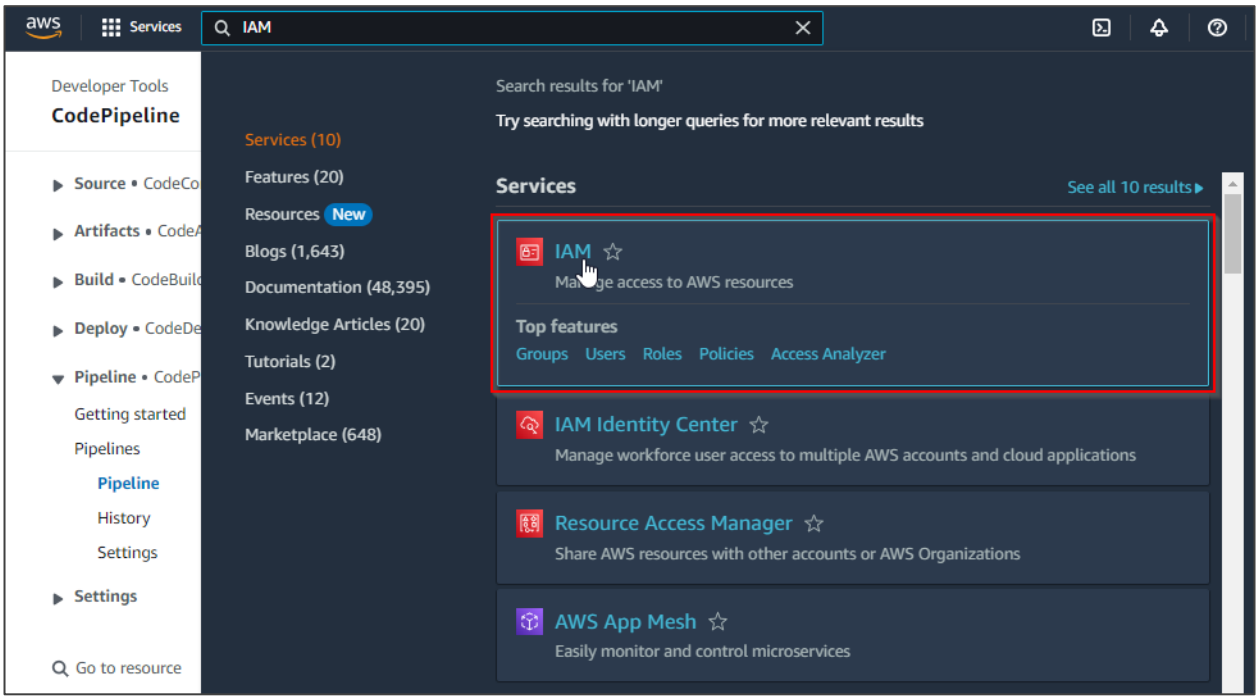
After creating the pipeline, the execution starts.



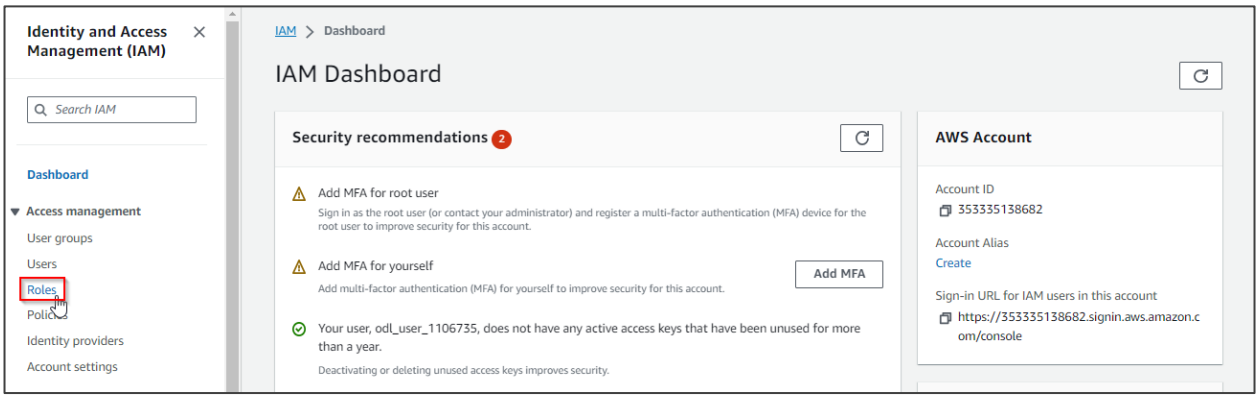
The first execution may fail because the service role for the pipeline may not have the necessary permissions to create S3 buckets.



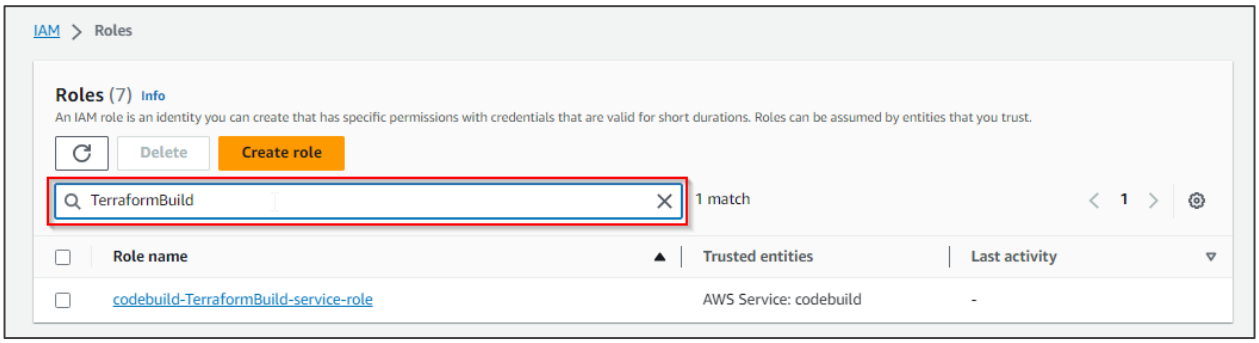
2.19 Search for and click on **IAM**



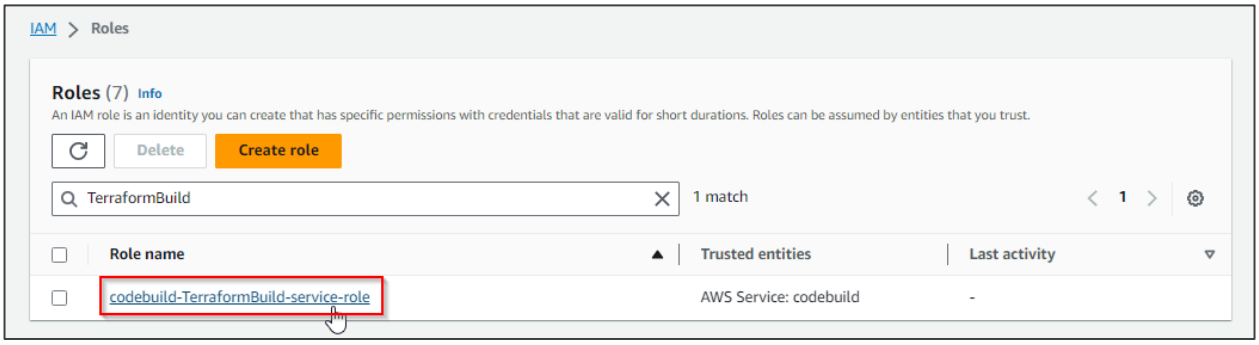
2.20 On the left side, click on **Roles**



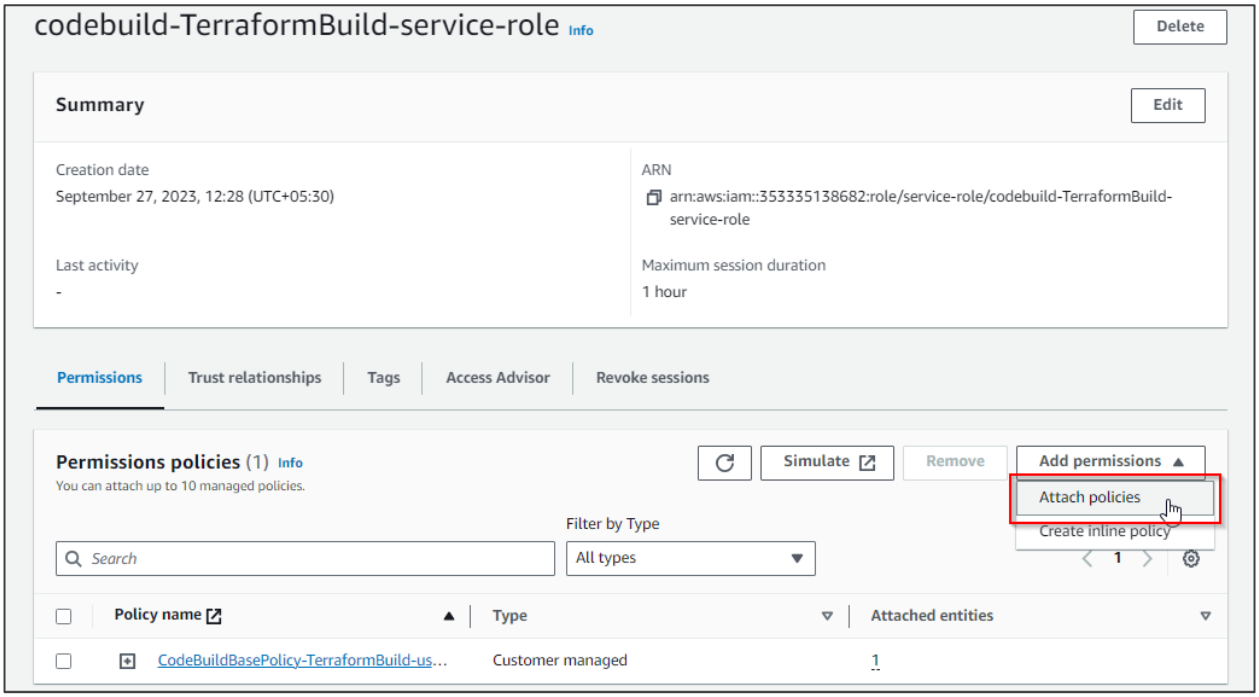
2.21 Search for **TerraformBuild**



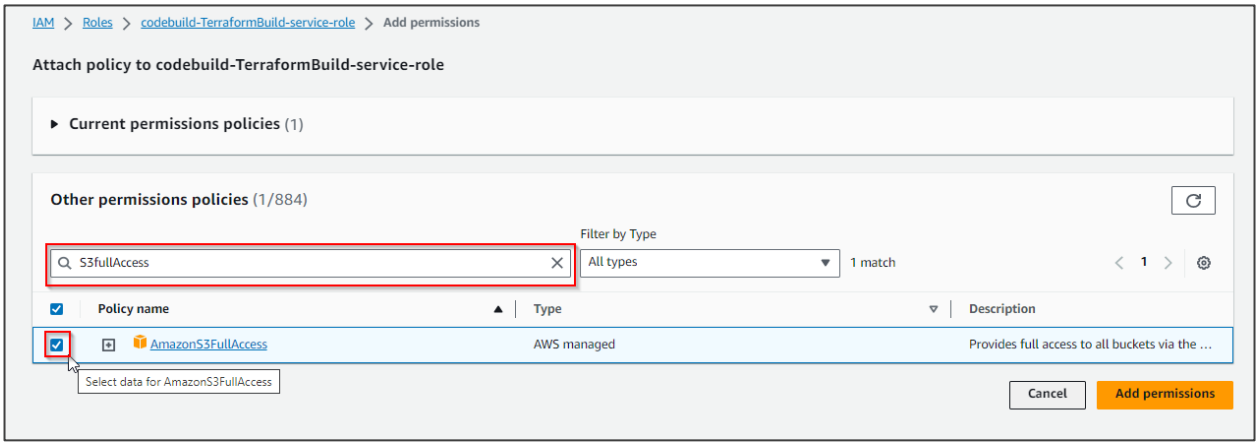
2.22 Click on the name of the role



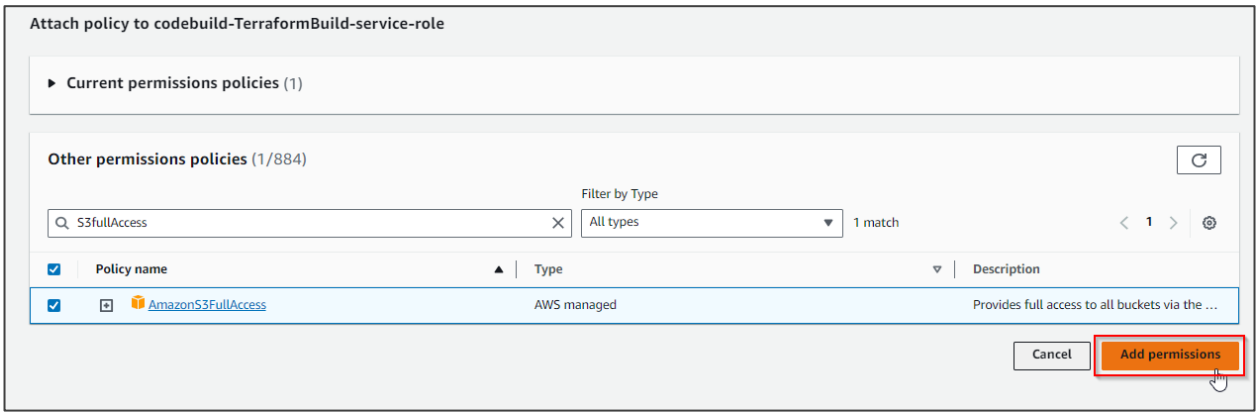
2.23 In the **Permissions policies** section, click on the **Add permissions** button and select the **Attach policies** option



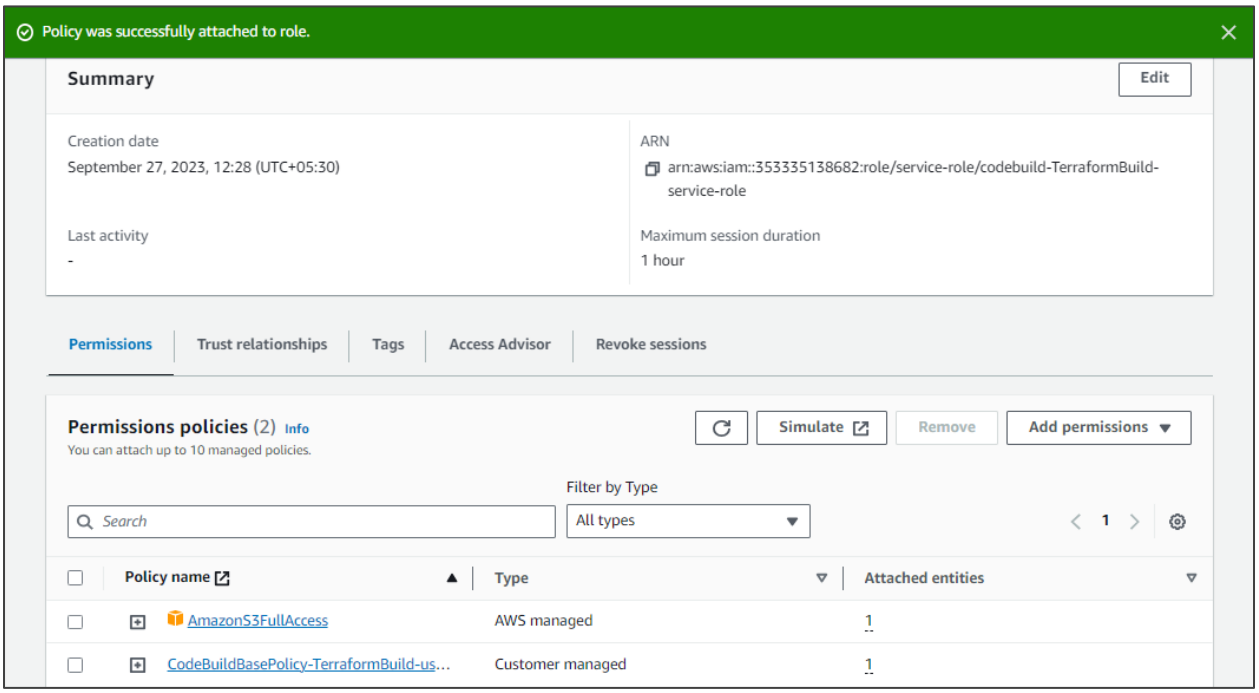
2.24 Search for **S3fullAccess** and click on the checkbox to add the policy



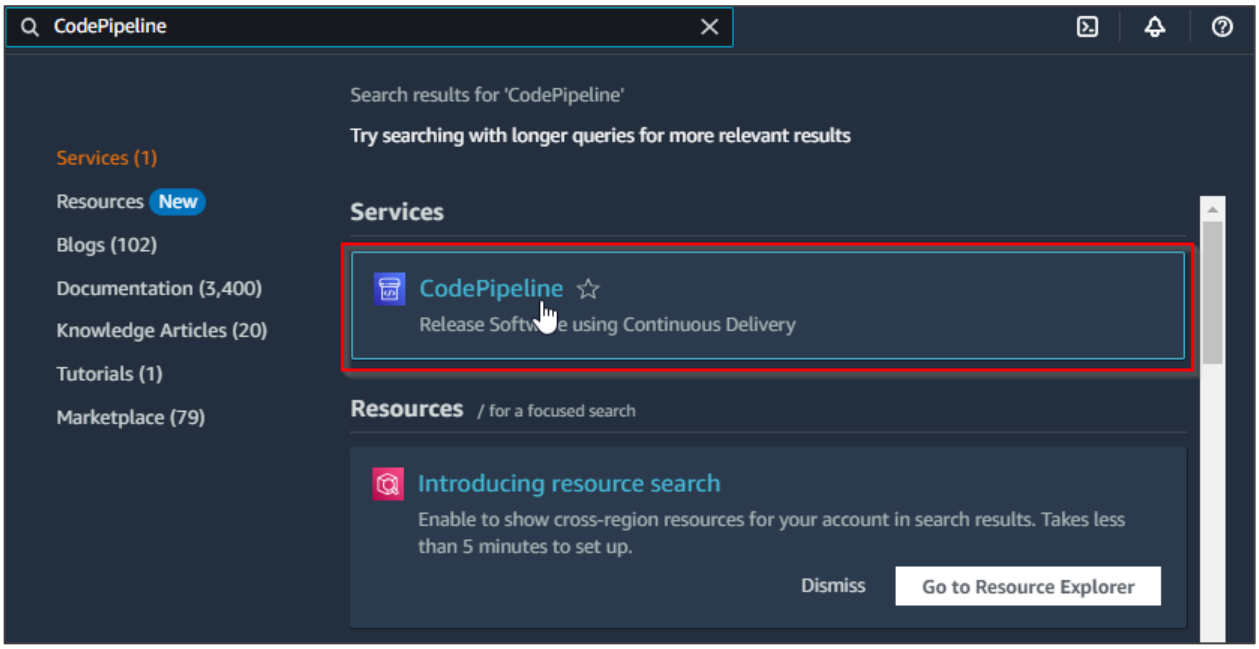
2.25 Click on the **Add permissions** button



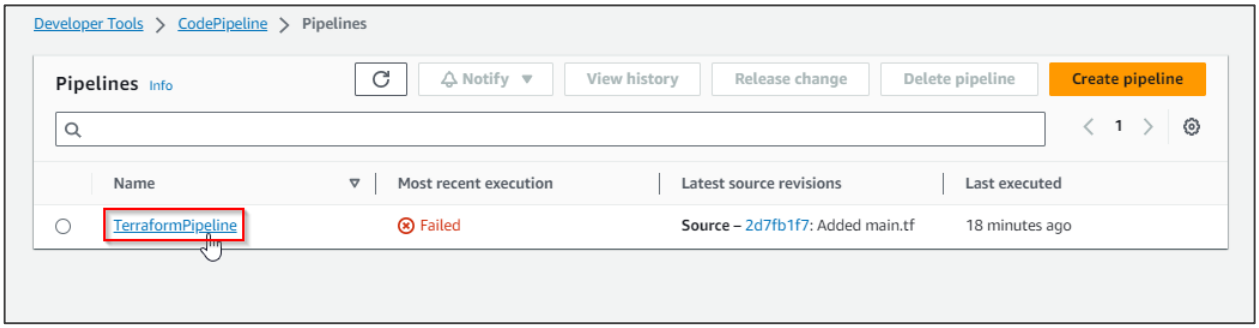
You can see that the policy is now successfully added to the role as shown in the screenshot below:



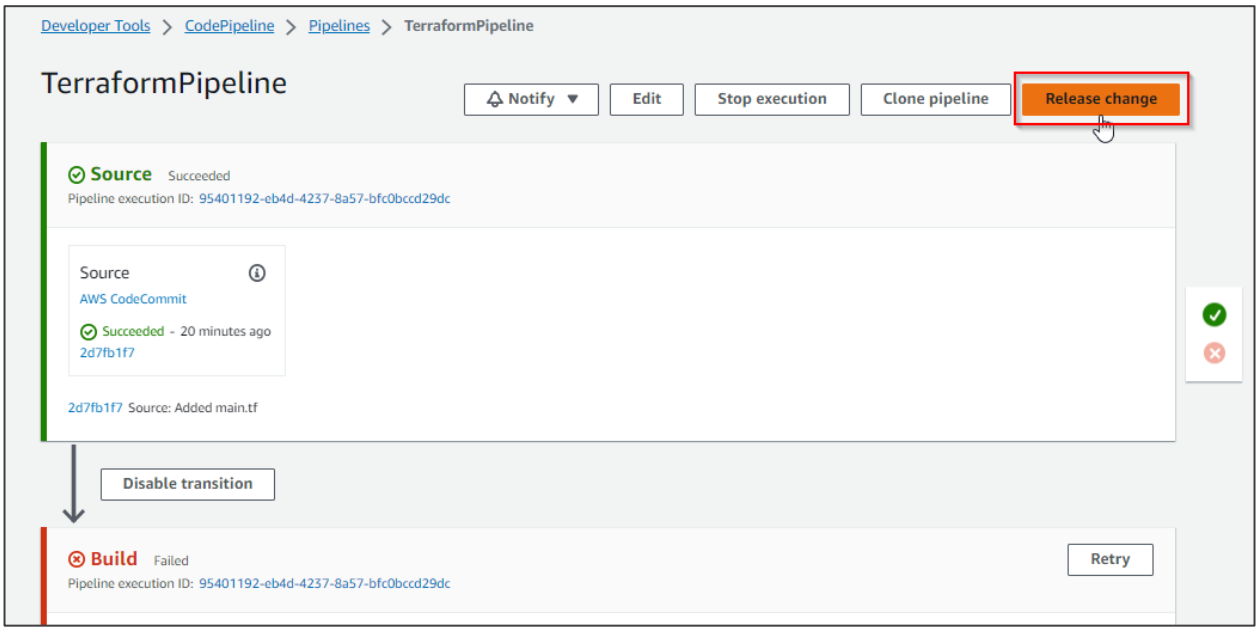
2.26 Now, search for and click on **CodePipeline**



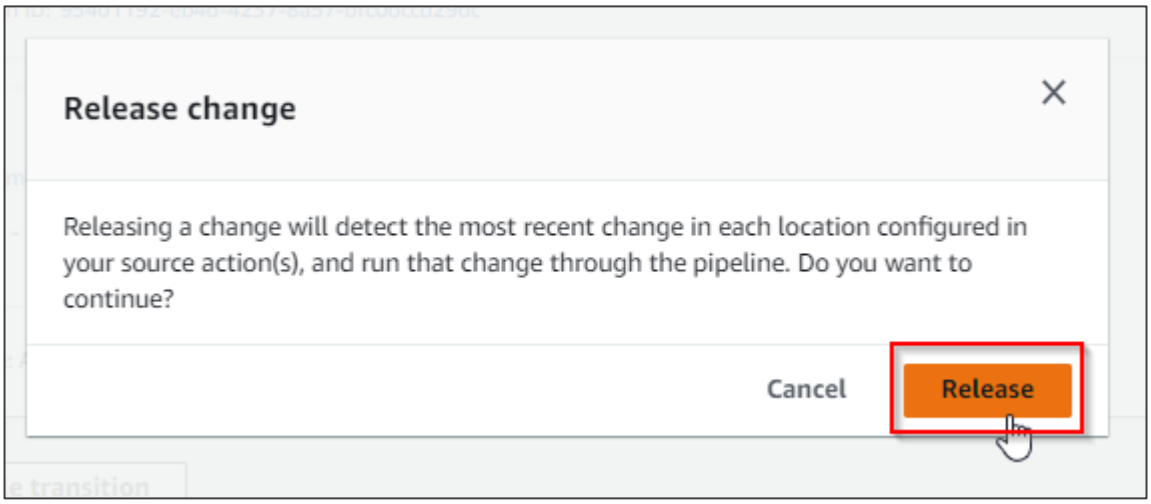
2.27 Click on the **Name** of the pipeline as shown in the screenshot below:



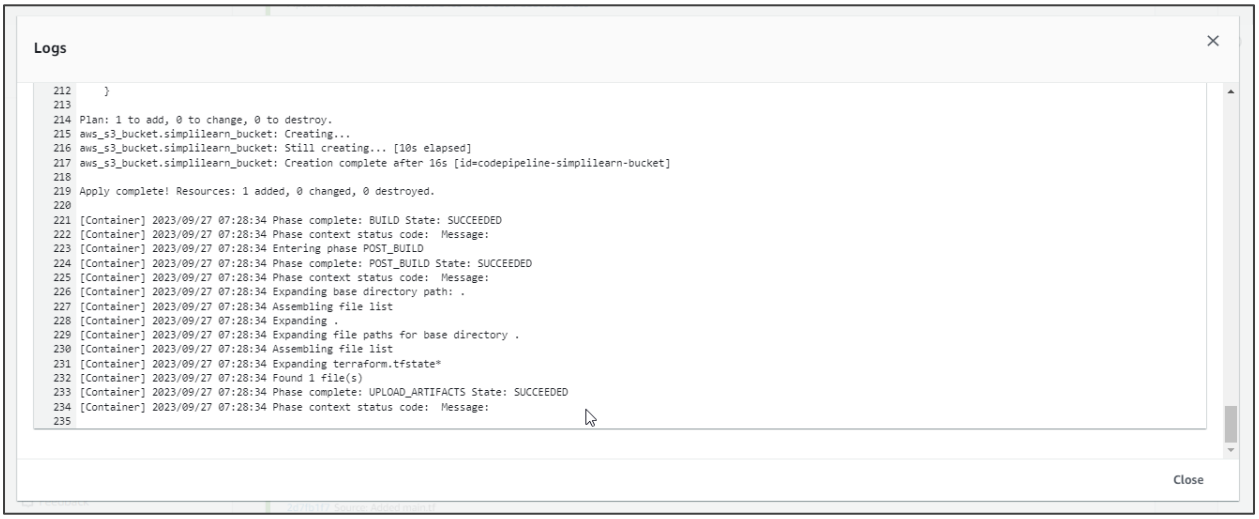
2.28 Click on the **Release change** button to invoke the pipeline



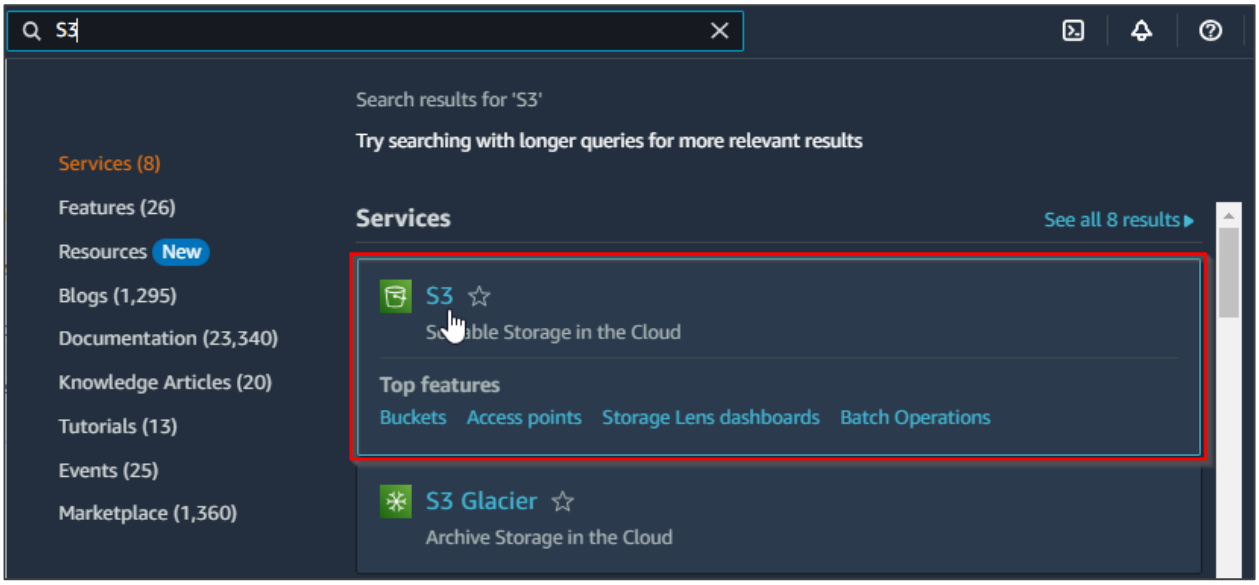
2.29 Click on the **Release** button



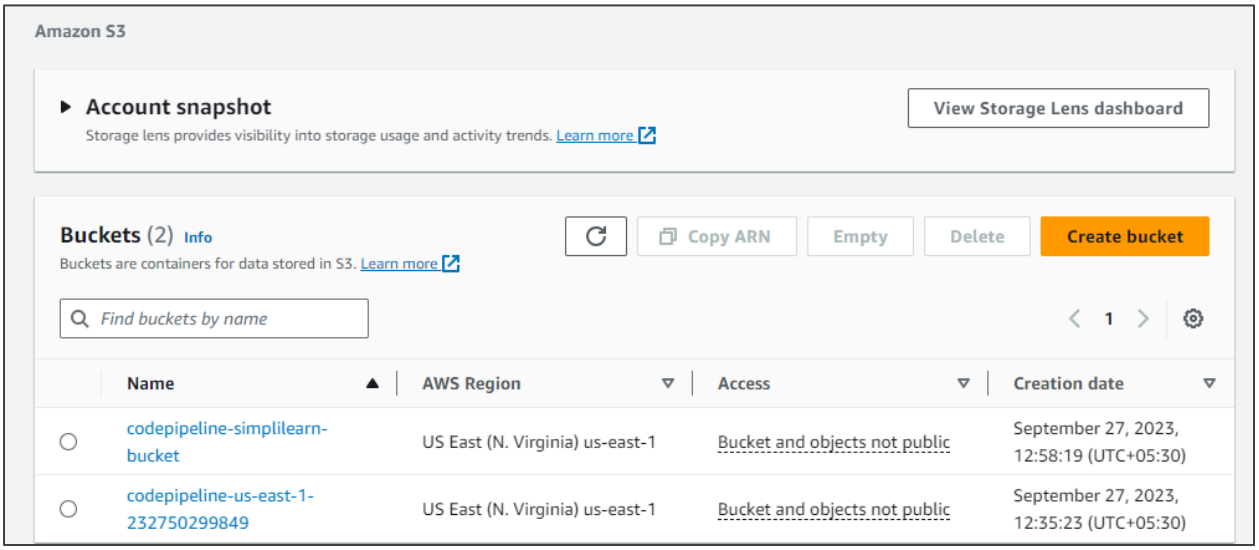
2.30 Once the build stage is completed, click on the **View logs** button in the **Build** section for validation



2.31 Now, search for and click on **S3** to validate whether the S3 bucket is successfully created in the S3 service



You can see that the S3 bucket is created successfully as shown in the screenshot below:



By following these steps, you have successfully completed the integration of Terraform script execution into AWS CodePipeline for efficient S3 bucket provisioning.